

Tolerância a Faltas

Apresentação Trabalho Prático

Diogo Marques, Francisco Ferreira, Ivan Ribeiro

Implementação do Algoritmo Raft

- Implementado em Rust.
- Ao executar, inicializa um nó *Maelstrom* que fica à escuta de mensagens do `stdin` e escreve para o `stdout`.

Teste com o *Maelstrom*

```
cargo build --release
java -jar maelstrom.jar test
  -w lin-kv
  --bin ".\target\release\pessi-raft.exe"
  --time-limit 60
  --node-count 3
  --concurrency 10n
  --rate 100
  --nemesis partition
  --nemesis-interval 3
```

Resultado do teste

> Everything looks good!

No entanto, não garante que o algoritmo esteja completamente correto.

O *Maelstrom* só testa um subconjunto limitado de cenários e não garante a cobertura de todos os estados possíveis ou falhas do sistema.

Verificação de Estados via Stateright

O que é o Stateright?

- Ferramenta de model checking para sistemas distribuídos escritos em Rust.
- Permite explorar exaustivamente os possíveis estados de um sistema e verificar se certas propriedades se mantêm.
- **Tudo a partir de código.**

O *model checking* com Stateright envolve a definição de:

- **Modelo:** Implementação do *Raft*, como modelo;
- **Ambiente:** Rede que não perde, duplica ou reordena mensagens;
- **Propriedades:**
 1. **Segurança de Eleição:** No máximo um líder deve poder ser eleito num mesmo termo.
 2. **Coerência do Log:** Se dois *logs* contêm uma entrada *committed* com o mesmo índice e termo, então as entradas são idênticas.
 3. **Vivacidade do Log:** O *log* não deverá ficar vazio eternamente.
 4. **Vivacidade de Eleições:** Um líder deverá ser eleito inevitavelmente.

Propriedades encontram-se implementadas em cima de código do algoritmo *Raft*.

Exemplo de Propriedade

```
fn election_liveness_property(state: &ActorModelState) -> bool {  
    state  
        .actor_states  
        .iter()  
        .any(|s| s.current_role() == Role::Leader)  
}
```

Exemplo de Uso

```
ActorModel::new((), ())  
  .actors(vec![RaftActor::new(...), RaftActor::new(...), ...])  
  .property(Sometimes, election_liveness_property)  
  .property(Sometimes, log_liveness_property)  
  .property(Always, election_safety_property)  
  .property(Always, log_safety_property)  
  .max_crashes(1)  
  .checker()  
  .target_max_depth(15)  
  ...
```

Demonstração e Mitigações de Faltas

API de Faltas: Injeção de código em pontos críticos da implementação do *Raft*.

- Conseguimos simular anomalias sem alterar o código do algoritmo.
- Avaliação de cada falha via testes unitários (cargo test).
- Verificar se as propriedades são violadas.

Identificação da Falta:

Um nó malicioso pode falsificar ou adulterar mensagens de outros nós.

Impacto:

Extremamente grave. Comprometerá completamente a segurança do *cluster*. Na prática:

- Derrubar eleições legítimas;
- Eleger-se a si próprio, ao manipular votos de outros nós;
- Causar divisões no *cluster*, enviando mensagens contraditórias para diferentes nós;
- Impedir propagação de comandos legítimos.

Demonstração do Impacto:

Demonstração pode ser feita alterando os campos onde é enviado o `node_id` da mensagem.

```
struct VoteRequestMessage {  
    node_id: Id,  
    current_term: TermId,  
    log_length: usize,  
    last_log_term: TermId,  
}
```

No *Stateright*, encontramos um caso que viola a propriedade de **Election Safety**.

Ambiente de Teste:

- Atores: Id(0): Malicioso, Id(1): Legítimo, Id(2): Legítimo

Sequência de Eventos:

1. Timeout(Id(1), ElectionTimeout)
2. Id(1) → VoteRequest { node_id: Id(1), current_term: 1, log_length: 0, last_log_term: 0 } → Id(2)
3. Id(2) → VoteResponse { node_id: Id(2), current_term: 1, vote_granted: true } → Id(1)
4. Timeout(Id(0), ElectionTimeout)
5. Id(0) → VoteResponse { node_id: Id(1), current_term: 1, vote_granted: true } → Id(0)

Estado Inválido Encontrado:

```
Node {  
  id: Id(0),  
  current_role: Leader,  
  current_term: 1,  
  ...  
}
```

```
Node {  
  id: Id(1),  
  current_role: Leader,  
  current_term: 1,  
  ...  
}
```

```
Node {  
  id: Id(2),  
  current_role: Follower,  
  current_term: 1,  
  ...  
}
```

Propriedades Violadas:

- Segurança da Eleição (Election Safety)

Medidas de Mitigação:

Assinaturas digitais.

Demonstração das Medidas de Mitigação:

Não relacionado com a unidade curricular.

Identificação da Falta:

Um nó malicioso pode votar em dois candidatos diferentes numa eleição no mesmo termo.

Impacto:

Permite que dois nós sejam eleitos como líder, o que quebra a propriedade de **Election Safety**.

Demonstração do Impacto:

Podemos injetar código na receção do `VoteRequest` e responder sempre positivamente ao voto, sem verificar se o nó já votou noutro candidato.

Enviar `VoteResponse` com `vote_granted`: `true` para qualquer `VoteRequest`.

Ambiente de Teste:

- Atores: Id(0): Malicioso, Id(1): Legítimo, Id(2): Legítimo

Sequência de Eventos:

1. Timeout(Id(0), ElectionTimeout)
2. Timeout(Id(2), ElectionTimeout)
3. Id(0) → VoteRequest { node_id: Id(0), current_term: 1, log_length: 0, last_log_term: 0 } → Id(1)
4. Id(0) → VoteRequest { node_id: Id(0), current_term: 1, log_length: 0, last_log_term: 0 } → Id(2)
5. Id(2) → VoteRequest { node_id: Id(2), current_term: 1, log_length: 0, last_log_term: 0 } → Id(0)
6. Id(0) → VoteResponse { node_id: Id(0), current_term: 1, vote_granted: true } → Id(2)
7. Id(1) → VoteResponse { node_id: Id(1), current_term: 1, vote_granted: true } → Id(0)

Estado Inválido Encontrado:

```
Node {  
  id: Id(0),  
  current_role: Leader,  
  current_term: 1,  
  ...  
}
```

```
Node {  
  id: Id(1),  
  current_role: Follower,  
  current_term: 1,  
  ...  
}
```

```
Node {  
  id: Id(2),  
  current_role: Leader,  
  current_term: 1,  
  ...  
}
```

Propriedades Violadas:

- Segurança da Eleição (Election Safety)

Medidas de Mitigação:

Mensagem adicional ElectedBy difundida no fim da eleição por todos os nós eleitos no fim da eleição, com a lista de nós que votaram neles.

Se um nó votou em mais de um candidato, este pode ser adicionado a uma *black-list*.

Novo problema: como garantir que um nó malicioso não possa alterar a lista de votantes?

Demonstração das Medidas de Mitigação:

Ambiente de Teste:

- Atores: Id(0): Malicioso, Id(1): L. com mitigação, Id(2): L. com mitigação

Propriedades a verificar:

- O nó malicioso é adicionado à *black-list*.

O nó malicioso ainda pode fazer com que outro nó se torne líder imediatamente após a eleição (Violação da propriedade de **Election Safety**).

Não há problema porque é invocada uma nova eleição de seguida. Mas não podemos verificar todas as propriedades.

Sequência de Eventos:

1. Timeout(Id(2), ElectionTimeout)
2. Id(2) → VoteRequest { node_id: Id(2), current_term: 1, log_length: 0, last_log_term: 0 } → Id(0)
3. Timeout(Id(1), ElectionTimeout)
4. Id(1) → VoteRequest { node_id: Id(1), current_term: 1, log_length: 0, last_log_term: 0 } → Id(0)
5. Id(2) → VoteRequest { node_id: Id(2), current_term: 1, log_length: 0, last_log_term: 0 } → Id(1)
6. Id(0) → VoteResponse { node_id: Id(0), current_term: 1, vote_granted: true } → Id(1)
7. Id(0) → VoteResponse { node_id: Id(0), current_term: 1, vote_granted: true } → Id(2)
8. Id(1) → Other(ElectedBy { leader: Id(1), term: 1, by: [Id(1), Id(0)] }) → Id(1)
9. Id(2) → Other(ElectedBy { leader: Id(2), term: 1, by: [Id(2), Id(0)] }) → Id(1)

Estado Inválido Encontrado:

```
Node {  
  id: Id(0),  
  blacklist: [],  
  current_role: Follower,  
  ...  
}
```

```
Node {  
  id: Id(1),  
  blacklist: [Id(0)],  
  current_role: Follower,  
  ...  
}
```

```
Node {  
  id: Id(2),  
  blacklist: [],  
  current_role: Leader,  
  ...  
}
```

(ElectedBy ainda não foi propagado para o nó 2)

Propriedades Verificadas:

- Nó malicioso (0) é adicionado à *black-list*.